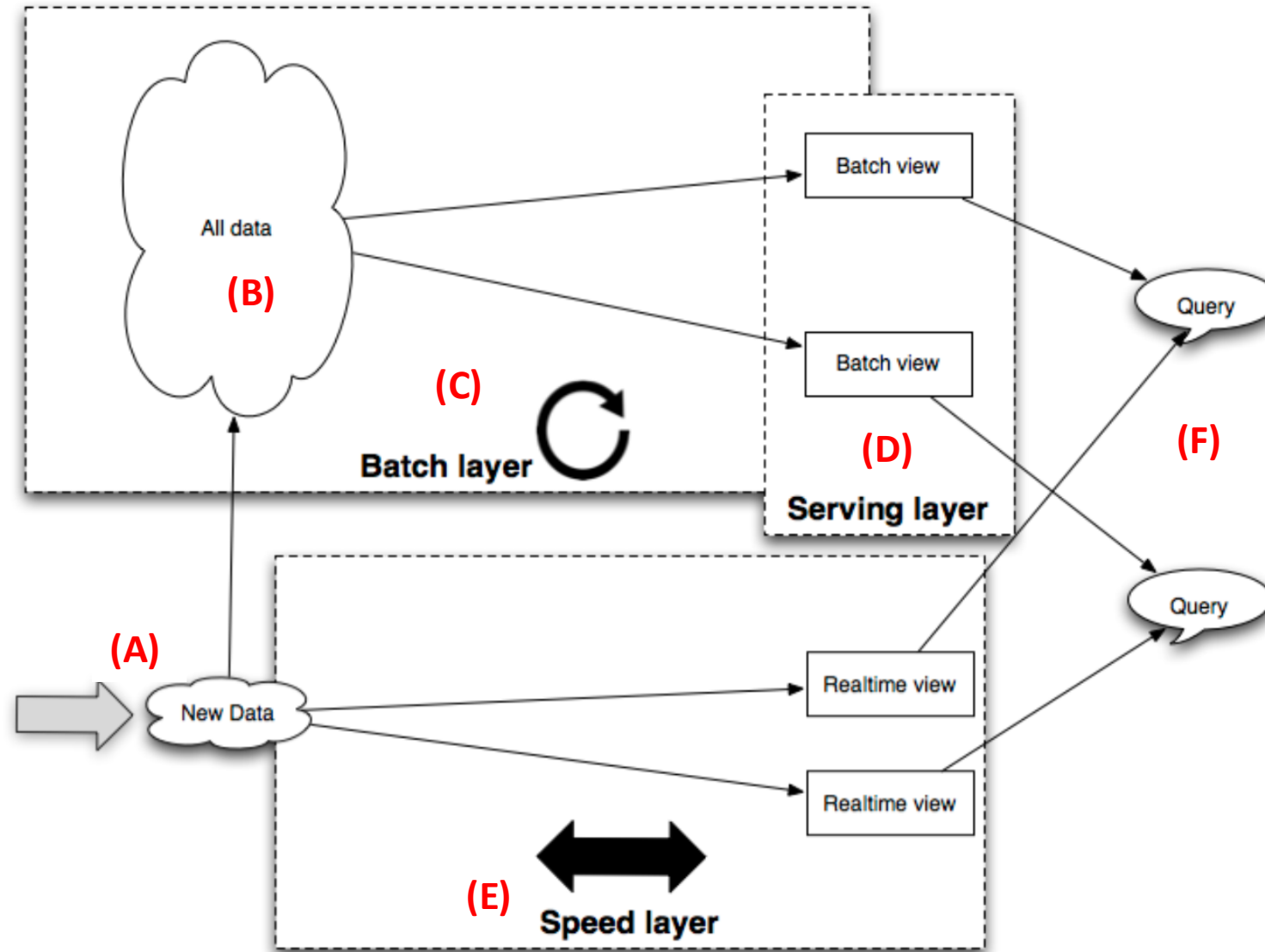




Information Systems for Big Data – 11th Lesson

Prof. G. Fenza gfenza@unisa.it

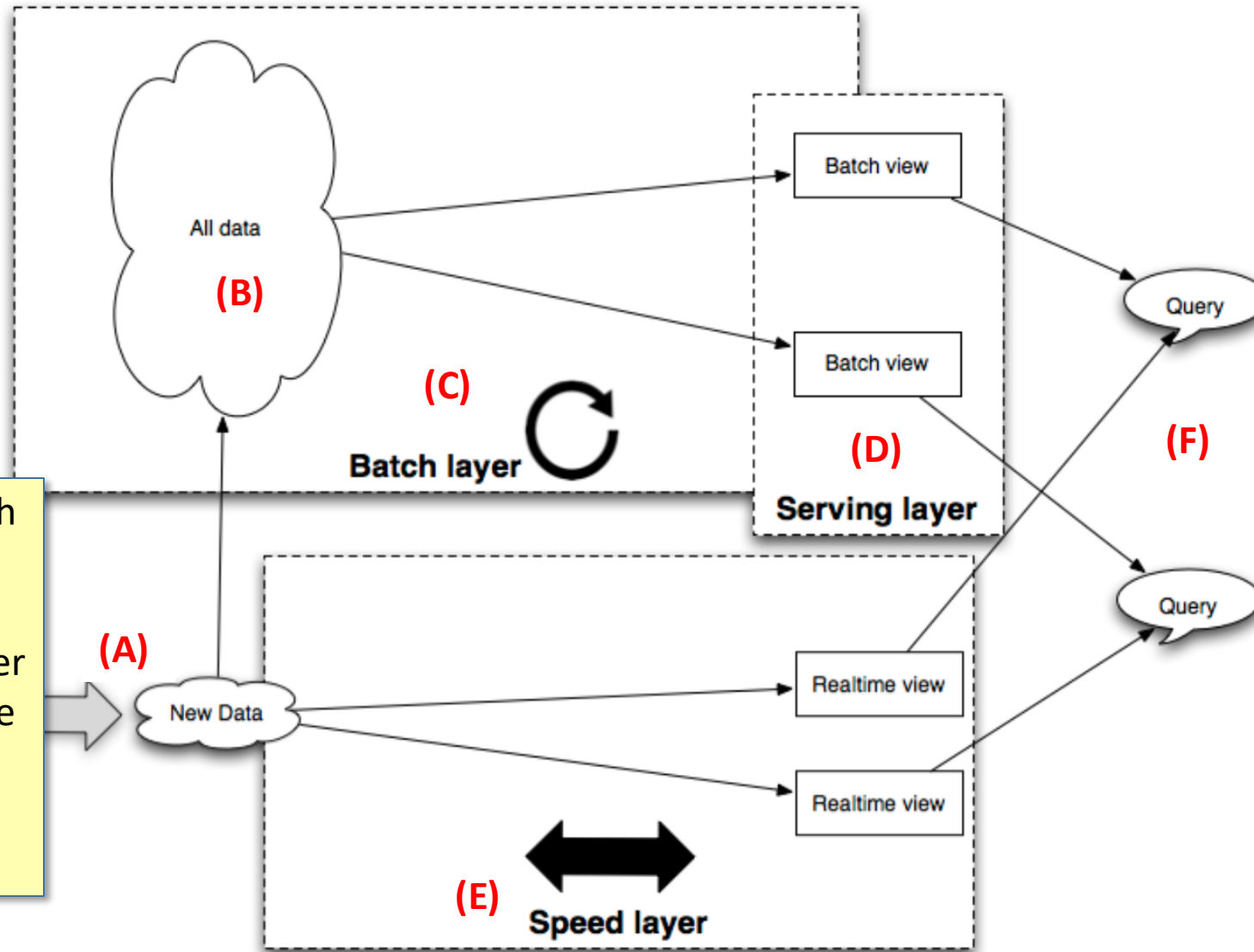
Lambda Architecture



Lambda Architecture



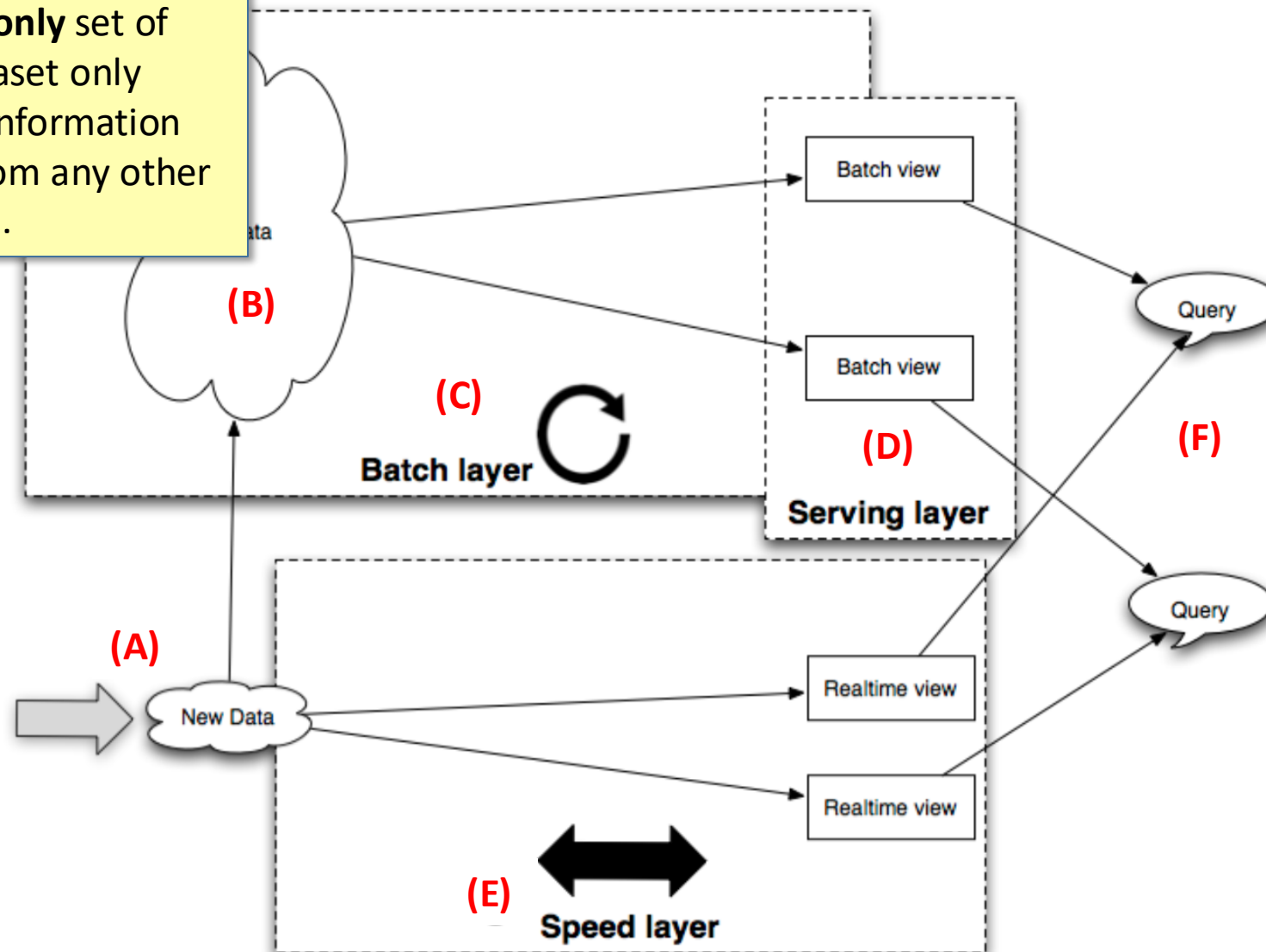
(A) All new data is sent to both the batch layer and the speed layer. In the batch layer, new data is appended to the master dataset. In the speed layer, the new data is consumed to do incremental updates of the realtime views.



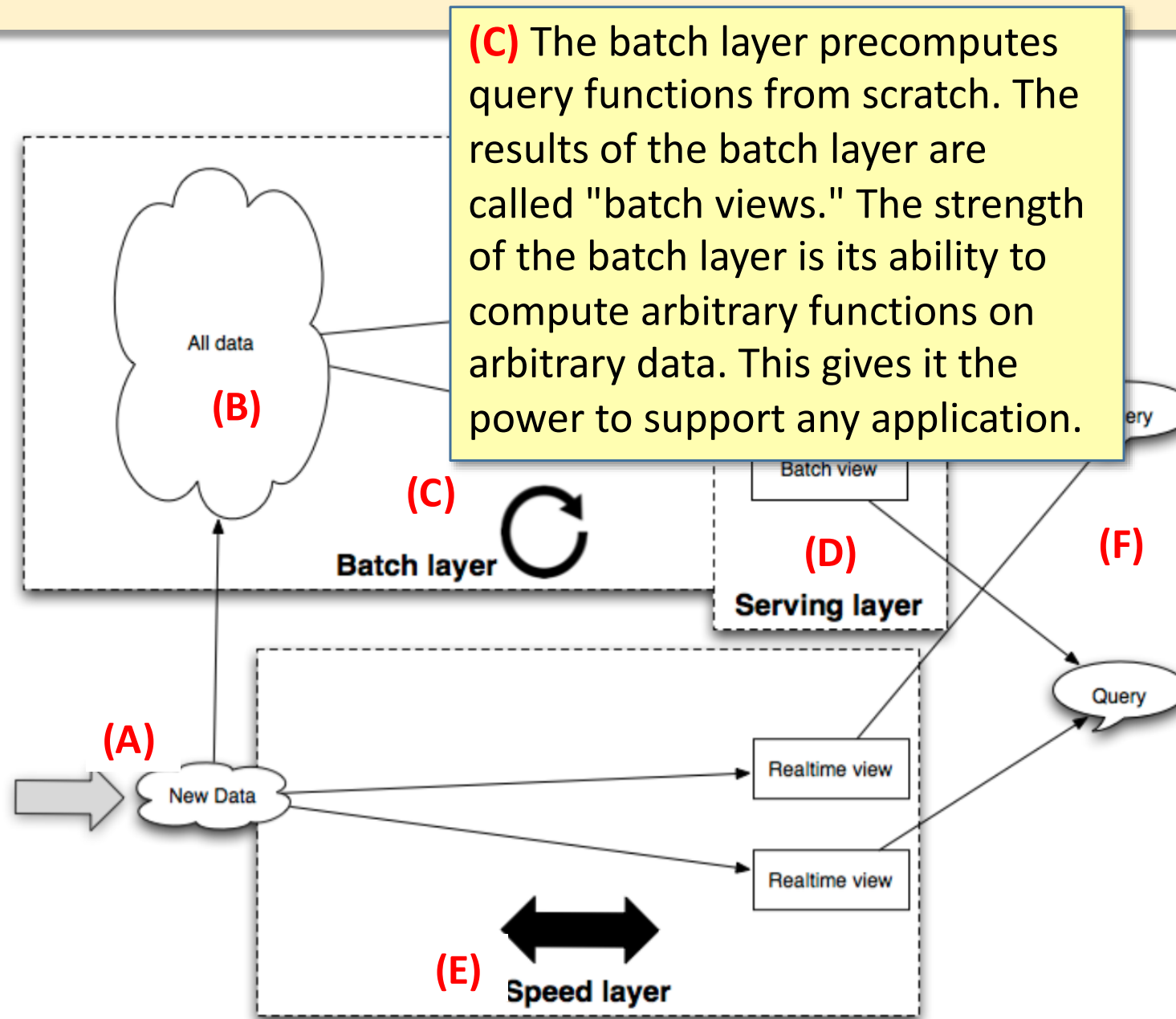
Lambda Architecture



(B) The master dataset is an **immutable, append-only** set of data. The master dataset only contains the **rawest** information that is not derived from any other information you have.



Lambda Architecture





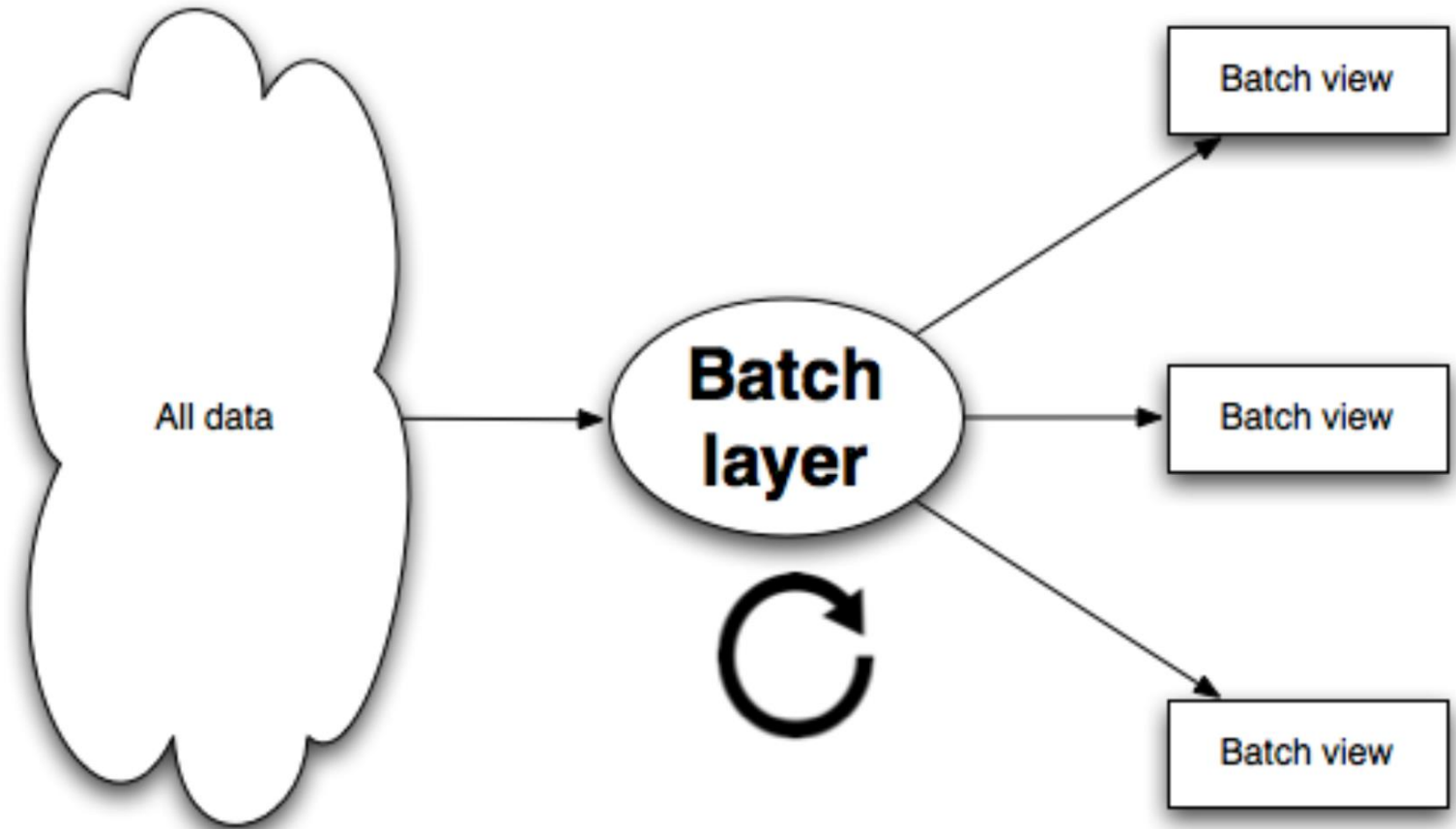
Lambda Architecture

Batch Layer – Distributed Batch Processing

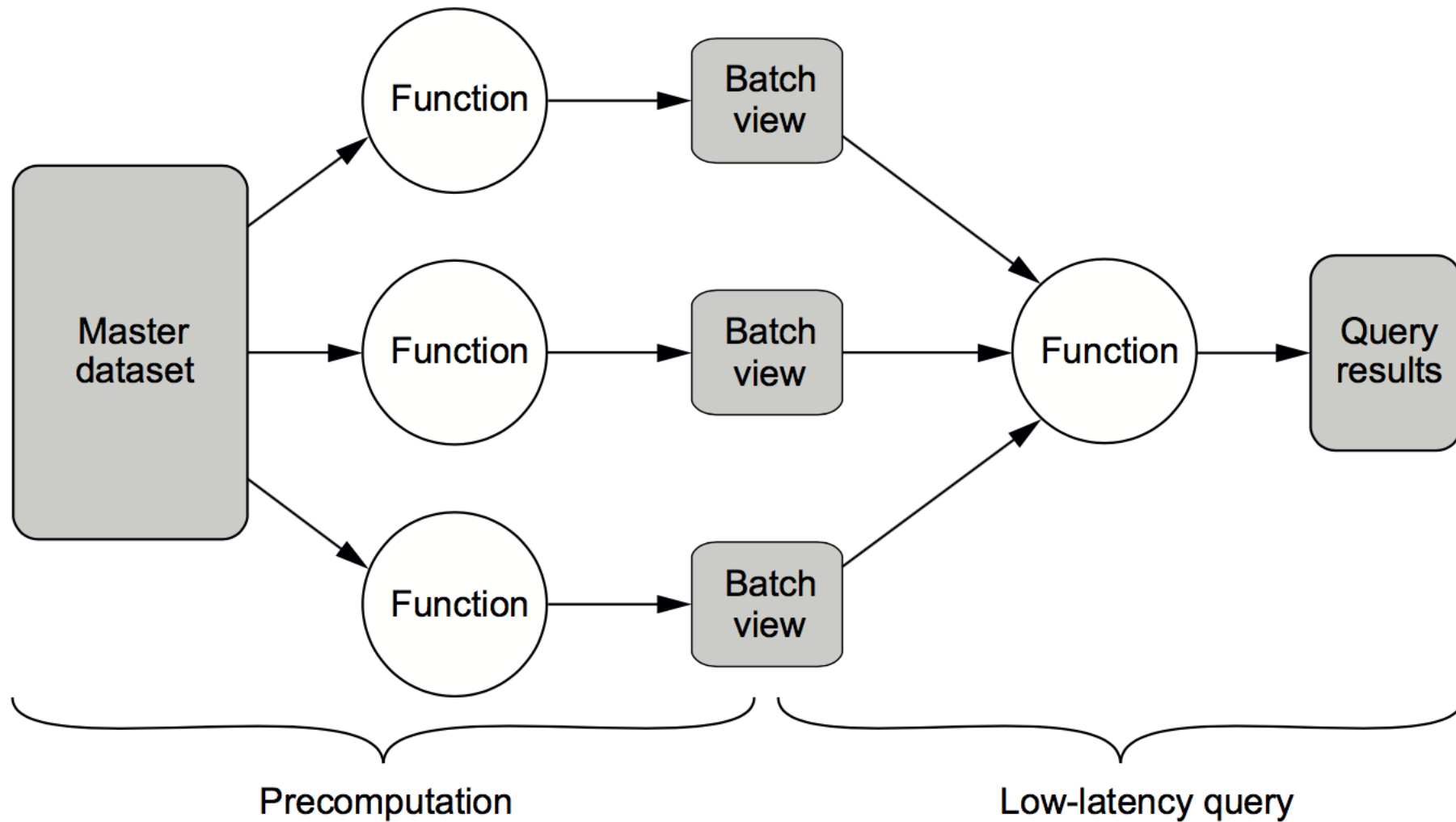
Batch Layer



1. Store Master Dataset
2. Compute Arbitrary Views



Precomputation of a query



Precomputing is not a panacea!

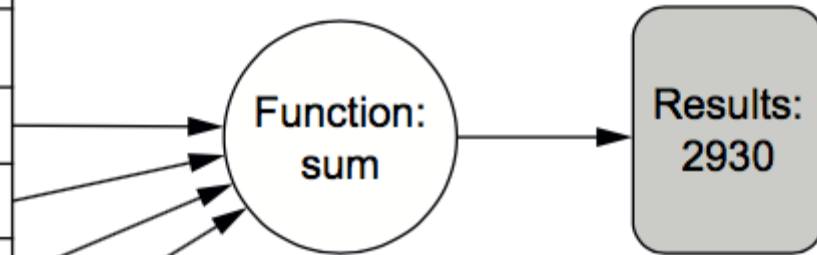


- Precomputing a query by running a function on the master dataset directly
- Splitting a query into pre-computation and on-the-fly components
- For instance, computing the number of pageviews by querying an indexed batch view

Precomputing is not a panacea!

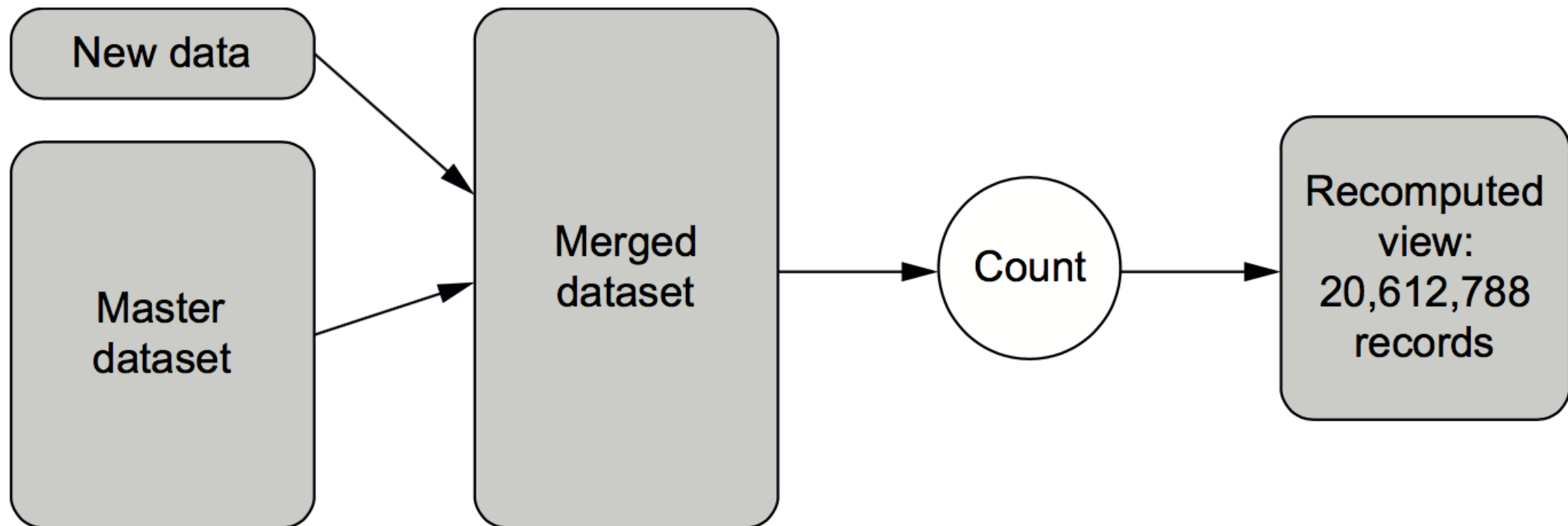


URL	Hour	# Pageviews
foo.com/blog	2012/12/08 15:00	876
foo.com/blog	2012/12/08 16:00	987
foo.com/blog	2012/12/08 17:00	762
foo.com/blog	2012/12/08 18:00	413
foo.com/blog	2012/12/08 19:00	1098
foo.com/blog	2012/12/08 20:00	657
foo.com/blog	2012/12/08 21:00	101

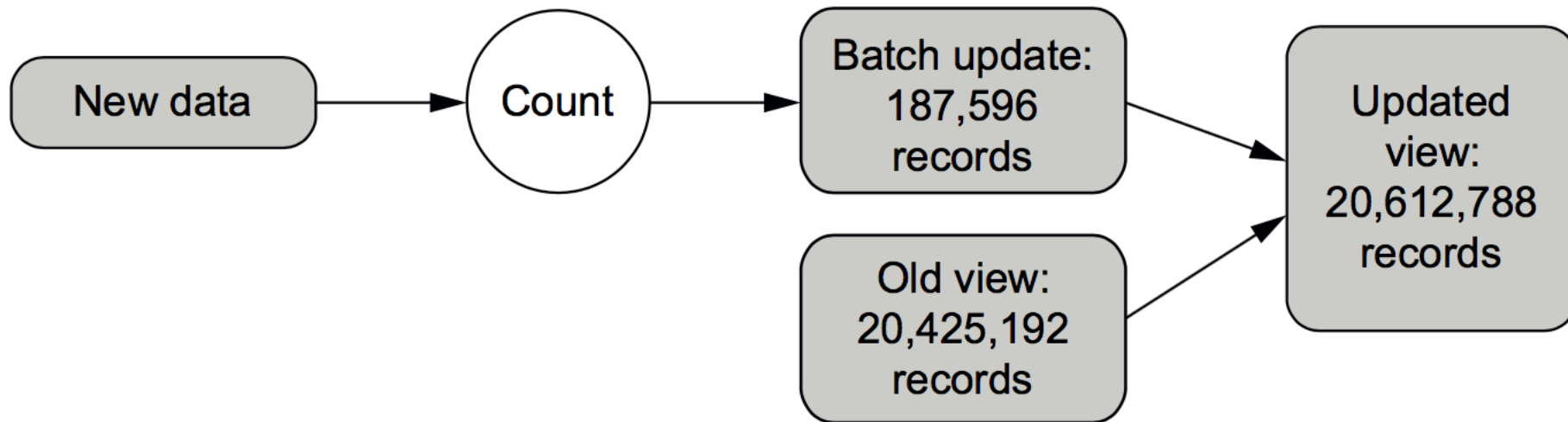


pageviews-over-time query as an example.

Recomputation algorithms



Incremental algorithms



Performance



URL	# Unique visitors
foo.com	2217
foo.com/blog	1899
foo.com/about	524
foo.com/careers	413
foo.com/faq	1212
...	...

Recomputation batch view

URL	# Unique visitors	Visitor IDs
foo.com	2217	1,4,5,7,10,12,14,....
foo.com/blog	1899	2,3,5,17,22,23,27,...
foo.com/about	524	3,6,7,19,24,42,51,...
foo.com/careers	413	12,17,19,29,40,42,...
foo.com/faq	1212	8,10,21,37,39,46,55,...
...	...	...

Incremental batch view

Recomputation algorithms vs Incremental algorithms



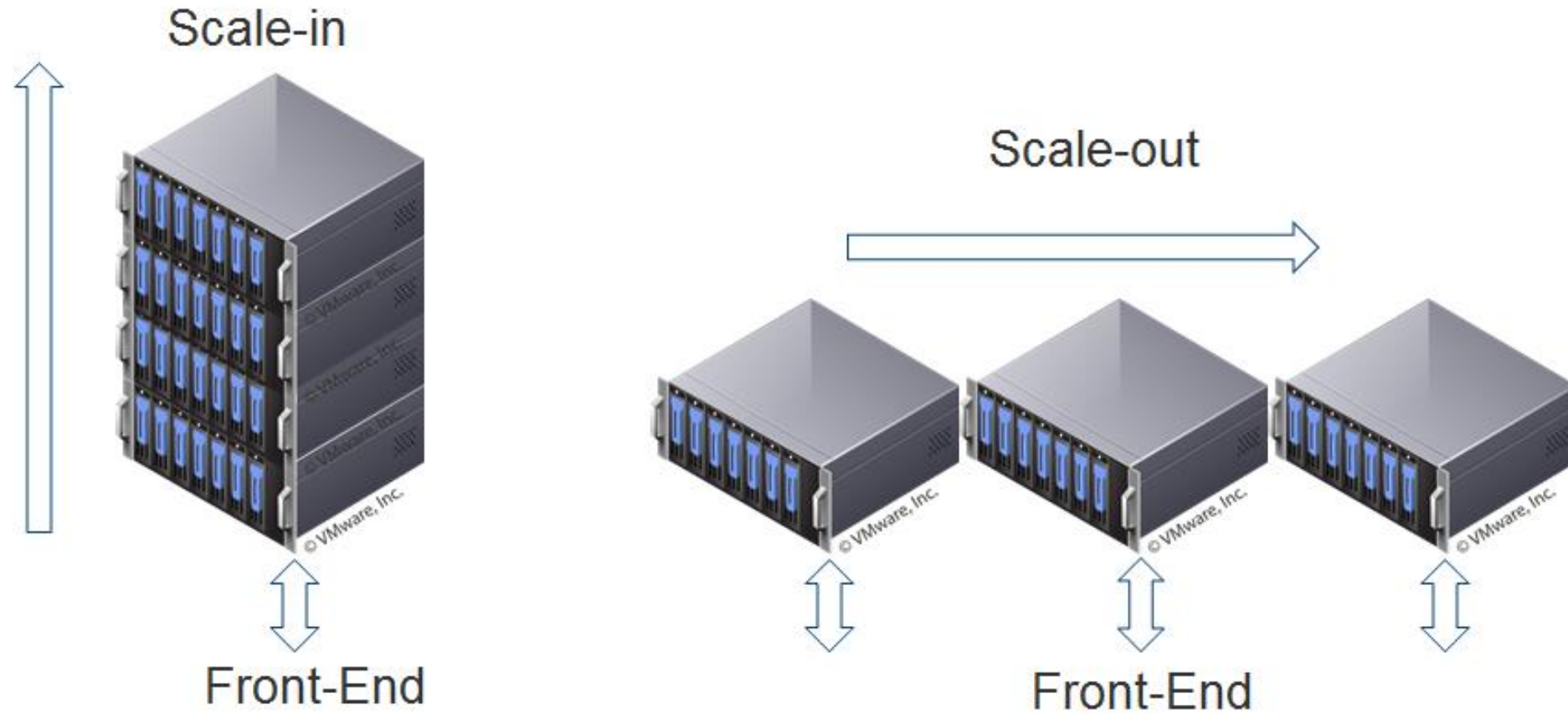
	Recomputation algorithms	Incremental algorithms
Performance	Requires computational effort to process the entire master dataset	Requires less computational resources but may generate much larger batch views
Human-fault tolerance	Extremely tolerant of human errors because the batch views are continually rebuilt	Doesn't facilitate repairing errors in the batch views; repairs are ad hoc and may require estimates
Generality	Complexity of the algorithm is addressed during precomputation, resulting in simple batch views and low-latency, on-the-fly processing	Requires special tailoring; may shift complexity to on-the-fly query processing
Conclusion	Essential to supporting a robust data-processing system	Can increase the efficiency of your system, but only as a supplement to recomputation algorithms

Distributed Processing – Cluster Computing Challenges



- Node failures
 - A single server can stay up for 3 years (1000 days)
 - 1000 servers in cluster → 1 failure/day
 - 1M servers in cluster → 1000 failures / day
- Network bottleneck
 - Network bandwidth = 1Gb
 - Moving 10TB takes approximatively 1 day
- Distributed programming is hard!
 - **Need a simple model that hides most of the complexity**

Distributed Processing – Cluster Computing Challenges



Last Lesson - Master Data Storage Requirements



Operation	Requisite	Discussion
Write	Efficient appends of new data	The only write operation is to add new pieces of data, so it must be easy and efficient to append a new set of data objects to the master dataset.
	Scalable storage	The batch layer stores the complete dataset—potentially terabytes or petabytes of data. It must therefore be easy to scale the storage as your dataset grows.
Read	Support for parallel processing	Constructing the batch views requires computing functions on the entire master dataset. The batch storage must consequently support parallel processing to handle large amounts of data in a scalable manner.
Both	Tunable storage and processing costs	Storage costs money. You may choose to compress your data to help minimize your expenses, but decompressing your data during computations can affect performance. The batch layer should give you the flexibility to decide how to store and compress your data to suit your specific needs.
	Enforceable immutability	It's critical that you're able to enforce the immutability property on your master dataset. Of course, computers by their very nature are mutable, so there will always be a way to mutate the data you're storing. The best you can do is put checks in place to disallow mutable operations. These checks should prevent bugs or other random errors from trampling over existing data.

- Il primo problema da risolvere è quello dei guasti hardware: non appena si iniziano a utilizzare molti pezzi di hardware, la possibilità che uno di essi si guasti è piuttosto alta.
- Un modo comune per evitare la perdita di dati è la replica: il sistema conserva copie ridondanti dei dati in modo che, in caso di guasto, sia disponibile un'altra copia.
- Questo è il modo in cui funziona il RAID, ad esempio, anche se Hadoop Distributed Filesystem (HDFS) adotta un approccio leggermente diverso.

Last Lesson - DFS

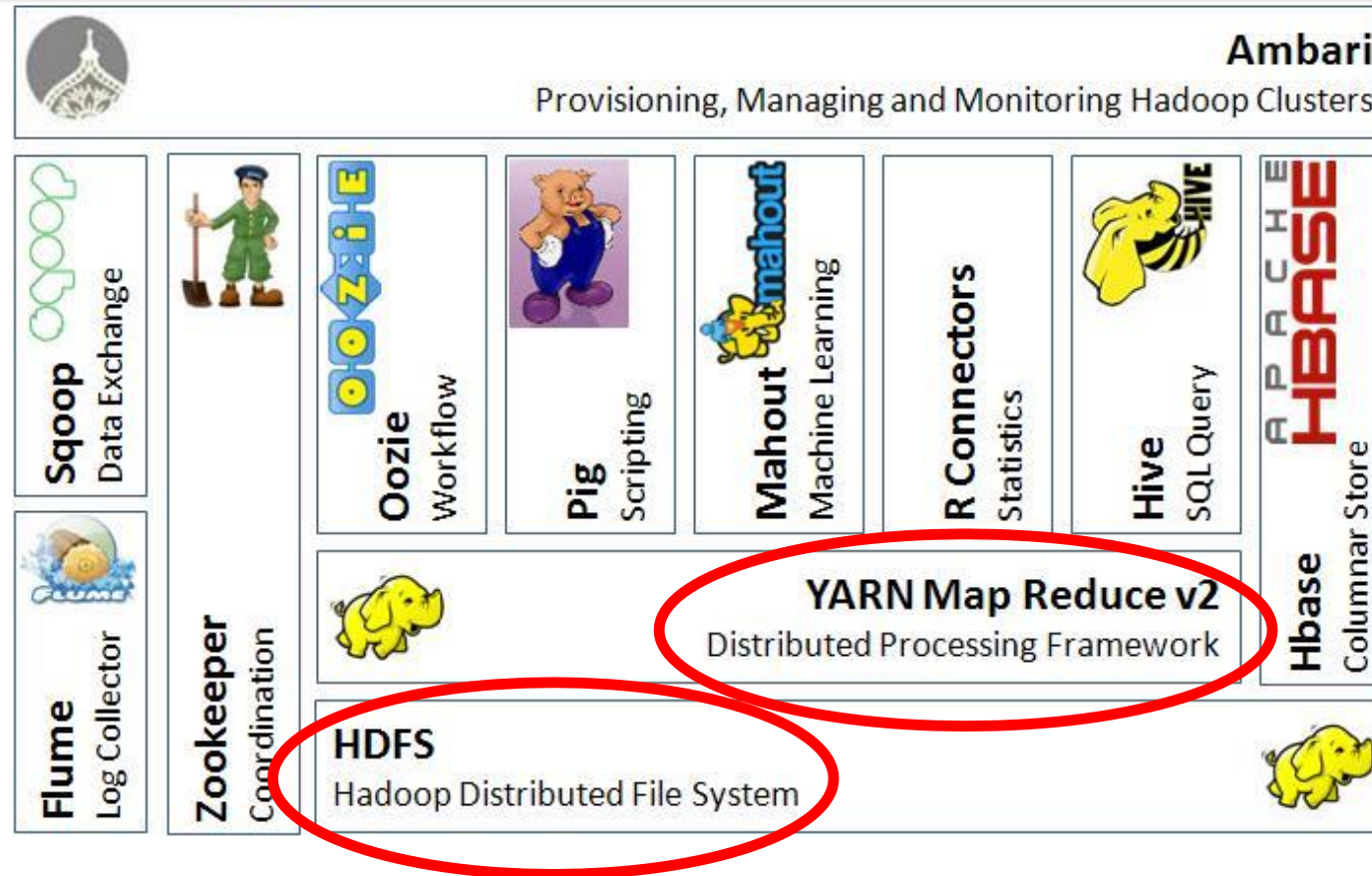


Operation	Requisite	Discussion
Write	Efficient appends of new data	Appending new data is as simple as adding a new file to the folder containing the master dataset.
	Scalable storage	Distributed filesystems evenly distribute the storage across a cluster of machines. You increase storage space and I/O throughput by adding more machines.
Read	Support for parallel processing	Distributed filesystems spread all data across many machines, making it possible to parallelize the processing across many machines. Distributed filesystems typically integrate with computation frameworks like MapReduce to make that processing easy to do (discussed in chapter 6).
Both	Tunable storage and processing costs	Just like regular filesystems, you have full control over how you store your data units within the files. You choose the file format for your data as well as the level of compression. You're free to do individual record compression, block-level compression, or neither.
	Enforceable immutability	Distributed filesystems typically have the same permissions systems you're used to using in regular filesystems. To enforce immutability, you can disable the ability to modify or delete files in the master dataset folder for the user with which your application runs. This redundant check will protect your previously existing data against bugs or other human mistakes.

- Il secondo problema è che la maggior parte delle attività di analisi deve essere in grado di combinare i dati in qualche modo, e i dati letti da un disco possono dover essere combinati con quelli di altri 99 dischi.
- Diversi sistemi distribuiti permettono di combinare i dati da più fonti, ma farlo correttamente è notoriamente difficile.
- MapReduce fornisce un modello di programmazione che astrae il problema dalle letture e scritture su disco, trasformandolo in un calcolo su insiemi di chiavi e valori.
- Il punto importante per la presente discussione è che ci sono due parti per il calcolo – map e reduce – ed è l'interfaccia tra le due parti dove avviene il "mix".
- Come HDFS, MapReduce ha un'affidabilità integrata.

- E' stata la prima soluzione per indirizzare le sfide del Cluster Computing:
 - Memorizzare i dati in modo ridondante su più nodi per garantire la persistenza e la disponibilità.
 - Spostare il calcolo vicino ai dati per ridurre al minimo lo spostamento dei dati stessi
 - Modello di programmazione semplice per nascondere la complessità di tutta questo processo

Hadoop Ecosystem



The real enabler for new processing models in Hadoop was the introduction of YARN (which stands for *Yet Another Resource Negotiator*) in Hadoop 2.

YARN is a cluster resource management system, which allows any distributed program (**not just MapReduce**) to run on data in a Hadoop cluster.



MapReduce

Coordination

Motivation for MapReduce



- Typical big data problem challenges:
 - How do we break up a large problem into smaller tasks that can be executed in **parallel**?
 - How do we assign tasks to workers distributed across a potentially **large number** of machines?
 - How do we ensure that the workers get the **data** they need?
 - How do we coordinate **synchronization** among the different workers?
 - How do we **share** partial results from one worker that is needed by another?
 - How do we accomplish all of the above in the face of software **errors** and hardware **faults**?

Motivation for MapReduce



- There was need for an abstraction that hides many system-level details from the programmer.
- MapReduce addresses this challenge by providing a simple abstraction for the developer, transparently handling most of the details behind the scenes in a ***scalable, robust, and efficient*** manner.
- MapReduce separates the ***what*** from the ***how***

Functional Programming



```
//In Java
double[] arr = new double[] {1,2,3};
double acc = 0;
for(int i=0; i<arr.length; i++) {
    acc += arr[i];
}
double ave = acc/arr.length;

System.out.println("The average of " + Arrays.toString(arr) + " is "+ave);
//> The average of [1.0, 2.0, 3.0] is 2.0
```

```
//In Scala
def list = List(1.0, 2.0 ,3.0)
def findAve(l: List[Double]): Double = l.reduce(_ + _)/l.length
findAve(List(1,2,3))

println(s"The average of $list is $ave")
//> The average of List(1.0, 2.0, 3.0) is 2.0
```

The Idea of MapReduce



- Inspired by the map and reduce functions in functional programming
- We can view map as a transformation over a dataset
 - This transformation is specified by the function f
 - Each functional application happens in **isolation**
 - The application of f to each element of a dataset can be parallelized in a straightforward manner
- We can view reduce as an aggregation operation
 - The aggregation is defined by the function g
 - Data locality: elements in the list must be “brought together”
 - If we can **group** elements of the list, also the reduce phase can proceed in parallel
- The framework coordinates the map and reduce phases:
 - Grouping intermediate results happens in parallel

Where the Magic Happens



- Implicit between the map and reduce phases is a **parallel “group by”** operation on intermediate keys
 - **Intermediate data arrive at each reducer sorted by the key**
 - **No ordering is guaranteed across reducers**
- Output keys from reducers are written back to HDFS
 - **The output may consist of r distinct files, where r is the number of reducers**
 - **Such output may be the input to a subsequent MapReduce phase**
- Intermediate keys (used in shuffle and sort) are transient:
 - **They are not stored on the distributed filesystem**
 - **They are “spilled” to the local disk of each machine in the cluster**

JobTracker and TaskTracker



- JobTracker

- JobTracker **determines the execution plan** by determining which files to process, **assigns** nodes to different **tasks**, and **monitors** all **tasks** as they're running.
- There is **only one JobTracker daemon per Hadoop cluster**. It's typically run on a server as a master node of the cluster.

- TaskTracker

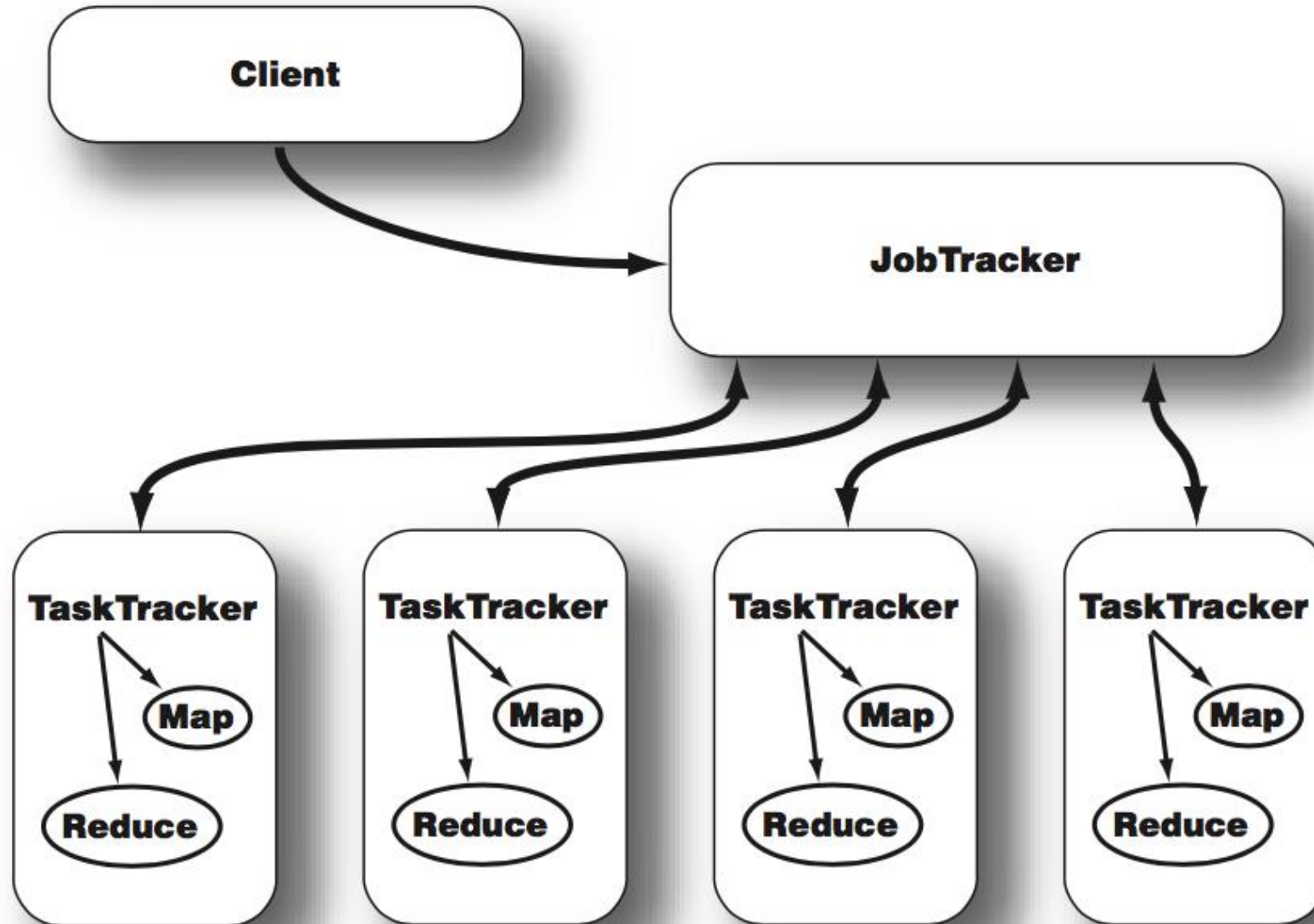
- The JobTracker is the master overseeing the overall execution of a MapReduce job and the TaskTrackers **manage the execution of individual tasks on each slave node**.
- One responsibility of the TaskTracker is to constantly communicate with the JobTracker.
- If the JobTracker fails to receive a heartbeat from a TaskTracker within a specified amount of time, it will assume the TaskTracker has crashed and will resubmit the corresponding tasks to other nodes in the cluster.

Coordination: Master

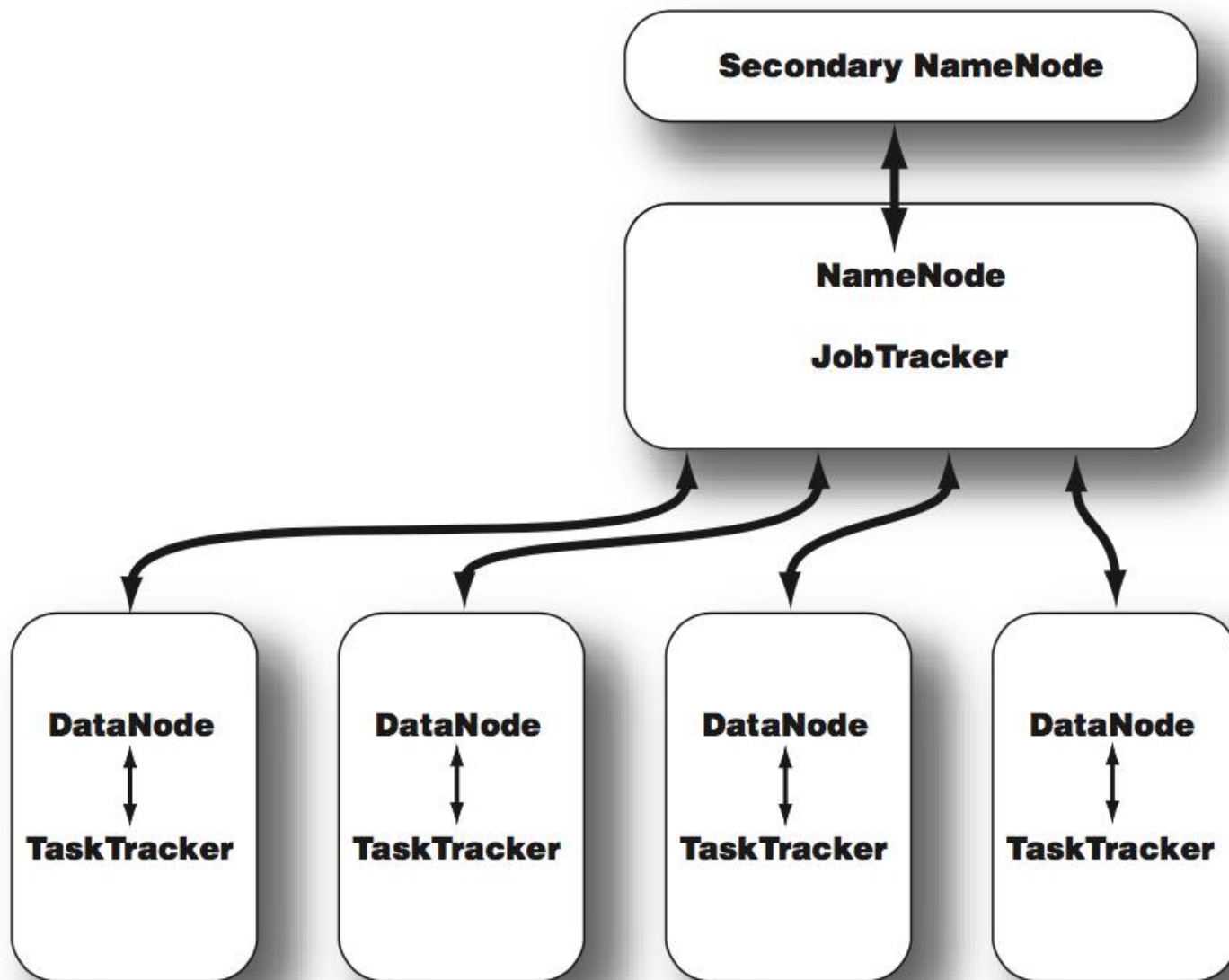


- Master node takes care of coordination:
 - Task status: (idle, in-progress, completed)
 - Idle tasks get scheduled as workers become available
 - **When a map task completes, it sends the master the location and sizes of its intermediate files, one for each reducer**
 - Master pushes this info to reducers
- Master pings workers periodically to detect failures

JobTracker and TaskTracker



Master/Slave Topology



Dealing with Failures



- Map worker failure
 - Its task is reassigned to another map worker
 - Reduce workers are notified when task is rescheduled on another worker
- Reduce worker failure
 - Its task is reassigned to another reduce worker
 - Reduce task is restarted (usually require restarting mapper tasks as well)
- Master failure
 - MapReduce task is aborted and client is notified
- Robust
 - Google's experience: **lost 1600 of 1800 machines once!**, but **finished fine**



MapReduce

Computational Model

Data Structures in MapReduce



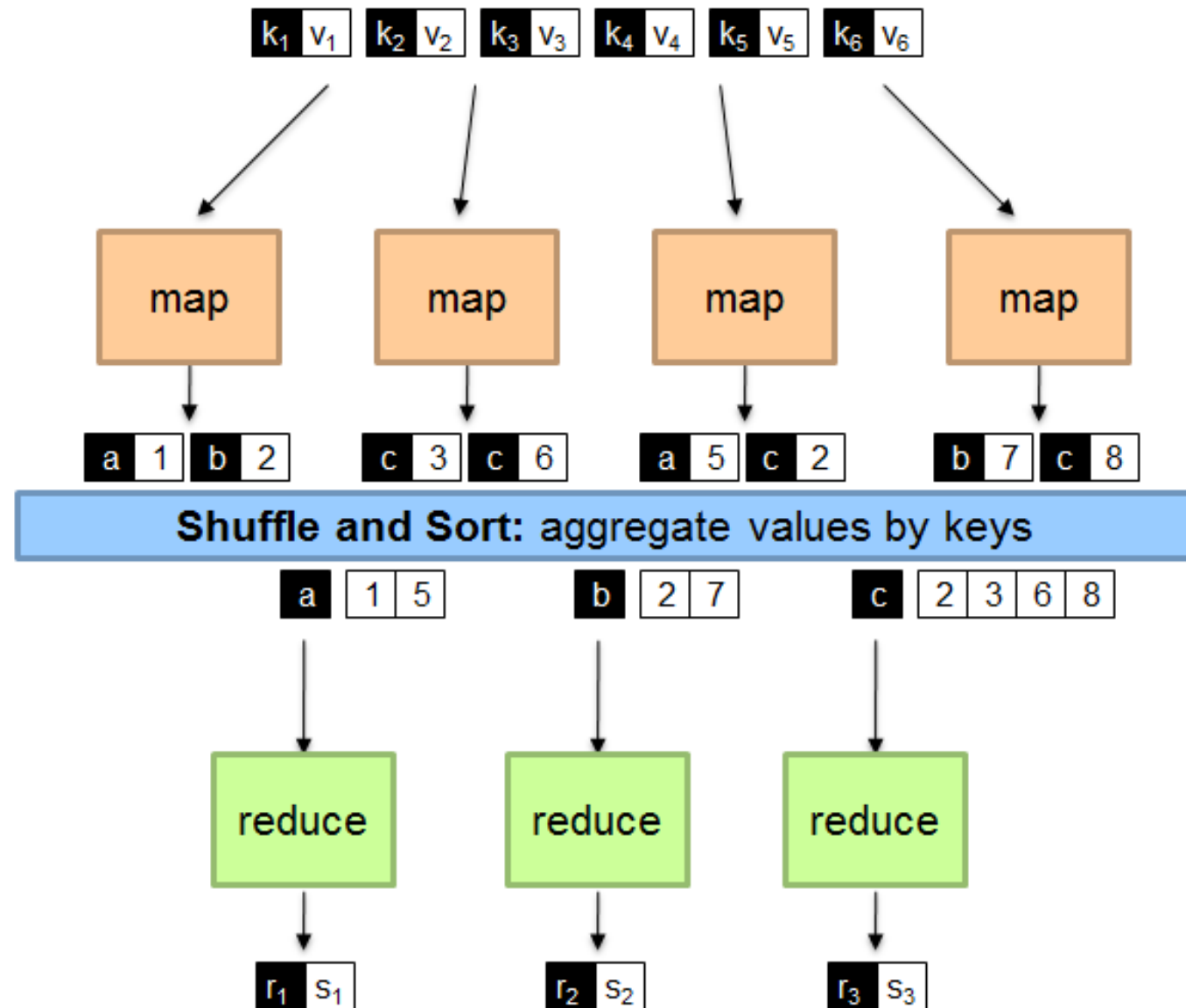
- Key-value pairs are the basic data structure in MapReduce
 - Keys and values can be: integers, float, strings, raw bytes
 - They can also be arbitrary data structures
- The design of MapReduce algorithms involves:
 - Imposing the key-value structure on arbitrary datasets
 - ❖ E.g.: for a collection of Web pages, input keys may be URLs and values may be the HTML content
 - In some algorithms, input keys are not used (e.g., wordcount), in others they uniquely identify a record
 - Keys can be combined in complex ways to design various algorithms

Map and Reduce Functions



- Programmers specify two functions:
 - **map** $(k_1, v_1) \rightarrow \text{list } [<k_2, v_2>]$
 - ❖ Map transforms the input into key-value pairs to process
 - list $[<k_2, v_2>]$ will be grouped according to key k_2 as $(k_2, \text{list } [v_2])$
 - **reduce** $(k_2, \text{list } [v_2]) \rightarrow [<k_3, v_3>]$
 - ❖ Reduce aggregates the list of values for each key
 - ❖ All values with the same key are sent to the same reducer
- The MapReduce environment takes in charge of everything else...
- A complex program can be decomposed as a succession of Map and Reduce tasks

A Brief View of MapReduce



Shuffle and Sort



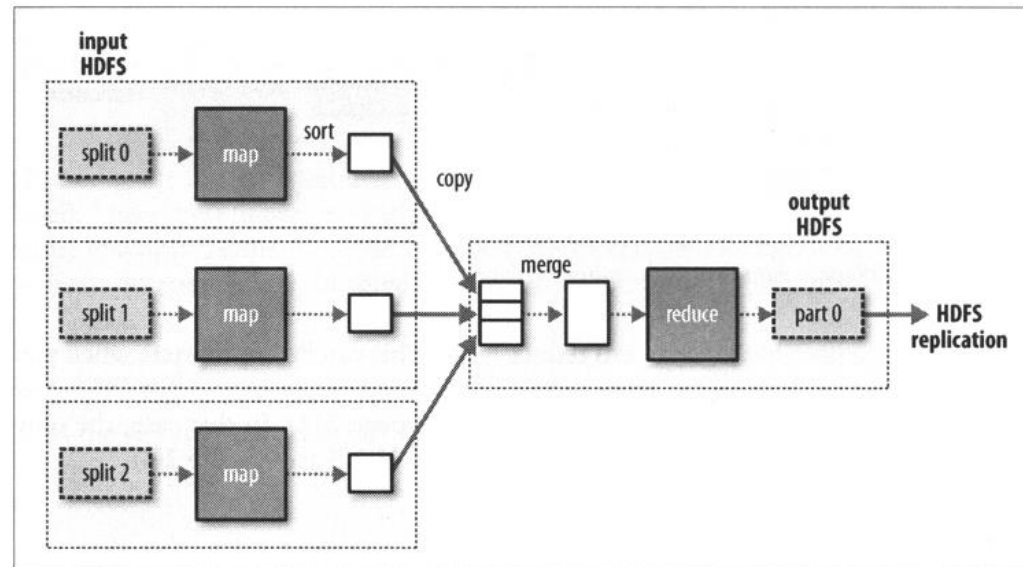
- **Shuffle**

- Transfers the map output from Mapper to a Reducer in MapReduce, via HTTP.

- **Sort**

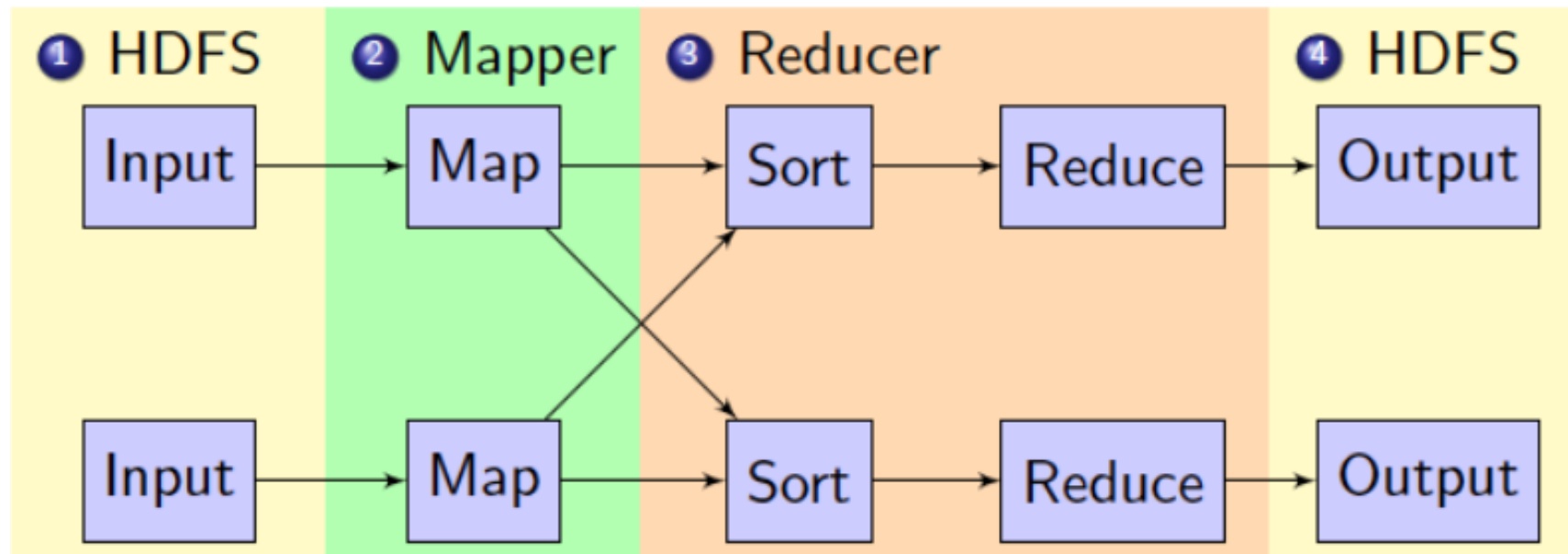
- The framework groups Reducer inputs by keys (since different Mappers may have output the same key) in this stage.
- Covers the merging and sorting of map outputs.

- Hadoop framework handles the Shuffle and Sort steps.



Hadoop MapReduce Brief Data Flow

- 1. Mappers read from HDFS
- 2. Map output is partitioned by key and sent to Reducers
- 3. Reducers sort input by key
- 4. Reduce output is written to HDFS
- **Intermediate results are stored on local FS of Map and Reduce workers**



“Hello World” in MapReduce



- Input:
 - key-value pairs: (docid, doc) of a file stored on the distributed filesystem
 - docid : unique identifier of a document
 - doc: is the text of the document itself
- Mapper:
 - Takes an input key-value pair, tokenize the line
 - Emits intermediate key-value pairs: the word is the key and the integer is the value
- The framework:
 - Guarantees all values associated with the same key (the word) are brought to the same reducer
- The reducer:
 - Receives all values associated to some keys
 - Sums the values and writes output key-value pairs: the key is the word and the value is the number of occurrences

Make It Running!



- Given two files:
 - file1: Hello World Bye World
 - file2: Hello Hadoop Goodbye Hadoop
- The first map emits:
 - < Hello, 1> < World, 1> < Bye, 1> < World, 1>
- The second map emits:
 - < Hello, 1> < Hadoop, 1> < Goodbye, 1> < Hadoop, 1>
- The output of the job is:
 - < Bye, 1> < Goodbye, 1> < Hadoop, 2> < Hello, 2> < World, 2>

Python requires Hadoop Streaming



```
import sys

for line in sys.stdin:
    line = line.strip()
    keys = line.split()
    for key in keys:
        value = 1
        print( "{}\t{}".format(key, value) )
```

Streaming means that runs over
StdInput and StdOutput Stream

```
import sys

last_key = None
running_total = 0

for input_line in sys.stdin:
    input_line = input_line.strip()
    this_key, value = input_line.split("\t", 1)

    try:
        value = int(value)
    except ValueError:
        continue

    if last_key == this_key:
        running_total += value
    else:
        if last_key:
            print( "{}\t{}".format(last_key, running_total) )
            running_total = value
            last_key = this_key

if last_key == this_key:
    print( "{}\t{}".format(last_key, running_total) )
```

Starting Docker Containers



- `docker pull effeerre/hadoop`
- `docker network create --driver bridge hadoop_network`
- `docker run -t -i -p 9864:9864 -d --network=hadoop_network --name=slave1 effeerre/hadoop`
- `docker run -t -i -p 9863:9864 -d --network=hadoop_network --name=slave2 effeerre/hadoop`
- `docker run -t -i -p 9862:9864 -d --network=hadoop_network --name=slave3 effeerre/hadoop`
- `docker run -t -i -p 9870:9870 -p 8088:8088 --network=hadoop_network --name=master -v $PWD:/data effeerre/hadoop`

Starting HDFS, MapReduce and YARN



- `hdfs namenode -format`
- `$HADOOP_HOME/sbin/start-dfs.sh`
- `$HADOOP_HOME/sbin/start-yarn.sh`
 - HDFS: `http://localhost:9870`
 - MapReduce Master: `http://localhost:8088/`

Upload data and Python Scripts



- `docker cp ./input1.txt master:/data/input1.txt`
- `docker cp ./input2.txt master:/data/input2.txt`
- `docker cp ./Exercises/Ex1MR/reducer.py master:/data/scripts/reducer.py`
- `docker cp ./Exercises/Ex1MR/mapper.py master:/data/scripts/mapper.py`

Change Privileges and Run Hadoop Streaming



- `hadoop dfs -chmod -R 1777 /`
- `chmod 7777 mapper.py`
- `chmod 7777 reducer.py`

- `mapred streaming -input /data -output /output2 -mapper ./mapper.py -reducer ./reducer.py`

Show the results: `hadoop dfs -cat /output2/part-00000`



MapReduce

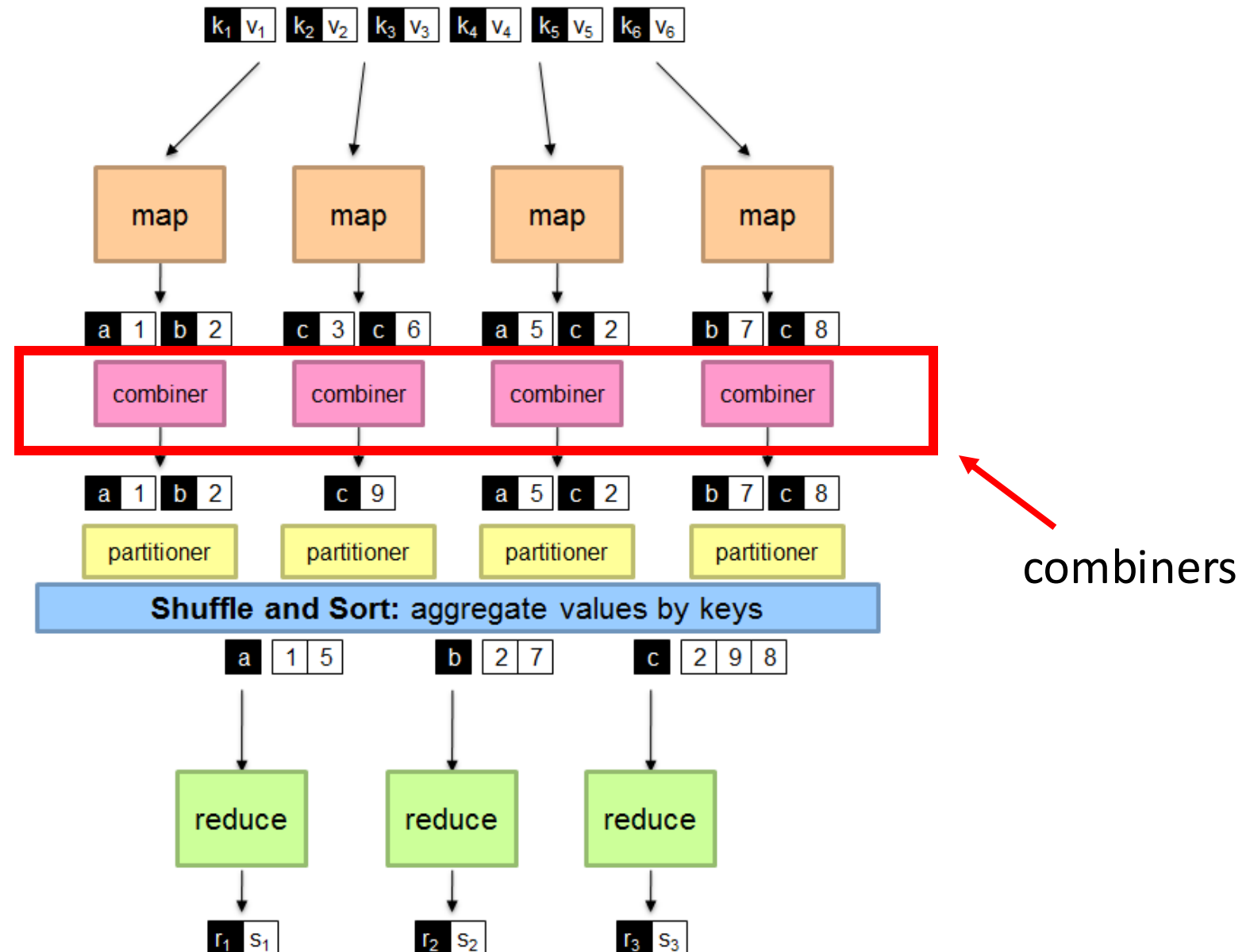
Refinements

Combiners



- Often a Map task will produce many pairs of the form $(k, v_1), (k, v_2), \dots$ for the same key k
 - E.g., popular words in the word count example
- Combiners are a general mechanism to reduce the amount of intermediate data, thus saving network time
 - They could be thought of as “mini-reducers”
- Warning!
 - The use of combiners must be thought carefully
 - ❖ Optional in Hadoop: the correctness of the algorithm **cannot depend on** computation (or even execution) of the combiners
 - ❖ A combiner operates on each map output key. It must have the same output key-value types as the Mapper class.
 - ❖ A combiner can produce summary information from a large dataset because it replaces the original Map output
 - Works only if reduce function is **commutative and associative**
 - ❖ In general, reducer and combiner **are not interchangeable**

Beyond Map and Reduce

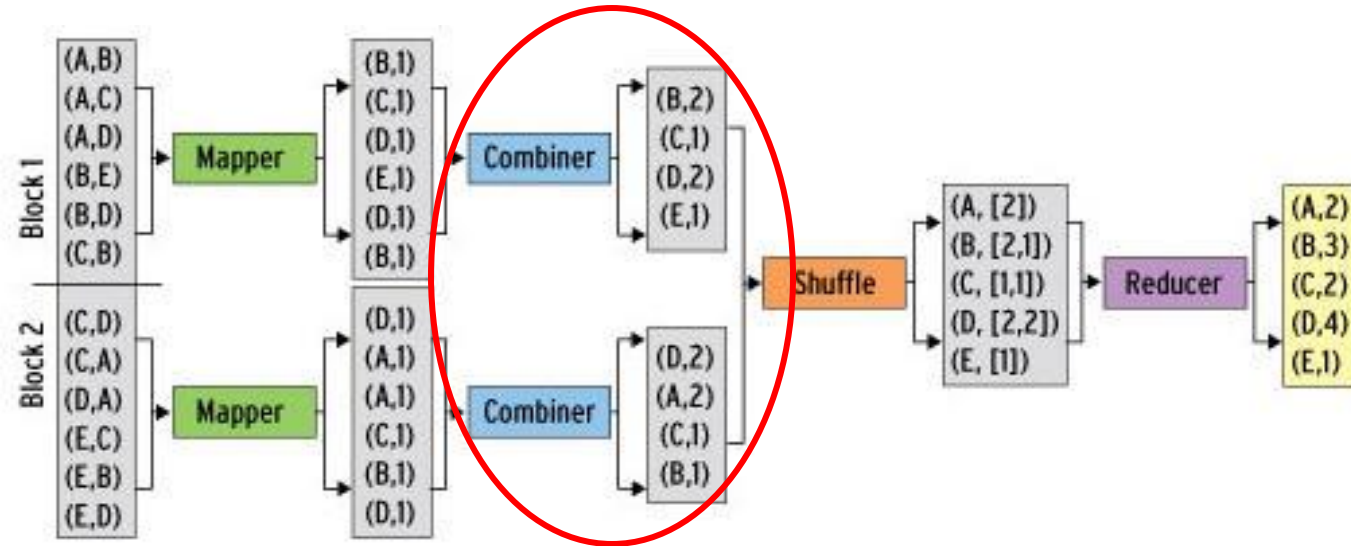


- We can use combiner only if the query we are implementing is:
 - **Commutative**
 - ❖ $a + b = b + a$
 - **Associative**
 - ❖ $(a + b) + c = a + (b + c)$
 - Examples: Sum, Product, and so on
 - Depending on the implementation: Average
 - Unfortunately, most functions don't work with Combiner trick: Median
- Because we are combining data in datanodes after the mapper execution and tuples with the same key may exist on different datanodes

Combiners in WordCount



- Combiner combines the values of all keys of **a single mapper node** (single machine):



- Much less data needs to be copied and shuffled!
- If combiners take advantage of all opportunities for local aggregation we have at most $m \times V$ intermediate key-value pairs
 - m : number of mappers
 - V : number of unique terms in the collection
- Note: not all mappers will see all terms

Combiners in WordCount



- In WordCount.java, you only need to add the follow line to Main:
job.setCombinerClass(IntSumReducer.class);
 - This is because in this example, **Reducer and Combiner do the same thing**
 - **Note: Most cases this is not true!**
 - You need to write an extra combiner class
- Given two files:
 - file1: Hello World Bye World
 - file2: Hello Hadoop Goodbye Hadoop
- The first map emits:
 - < Hello, 1> < World, 2> < Bye, 1>
- The second map emits:
 - < Hello, 1> < Hadoop, 2> < Goodbye, 1>

Partitioner



- Partitioner controls the partitioning of the keys of the intermediate map-outputs.
 - The key (or a subset of the key) is used to derive the partition, typically by a *hash function*.
 - The total number of partitions is the same as the number of reduce tasks for the job.
 - ❖ This controls which of the m reduce tasks the intermediate key (and hence the record) is sent to for reduction.
- System uses HashPartitioner by default:
 - $\text{hash}(\text{key}) \bmod R$
- Sometimes useful to override the hash function:
 - E.g., ***$\text{hash}(\text{hostname}(\text{URL})) \bmod R$*** ensures URLs from a host end up in the same output file
 - ❖ <https://www.unsw.edu.au/faculties> and <https://www.unsw.edu.au/about-us> will be stored in one file
- Job sets Partitioner implementation (in Main)

MapReduce: Recap



- Programmers must specify:
 - $\text{map}(k_1, v_1) \rightarrow [(k_2, v_2)]$
 - $\text{reduce}(k_2, [v_2]) \rightarrow [\langle k_3, v_3 \rangle]$
 - All values with the same key are reduced together
- Optionally, also:
 - $\text{combine}(k_2, [v_2]) \rightarrow [\langle k_3, v_3 \rangle]$
 - ❖ Mini-reducers that run in memory after the map phase
 - ❖ Used as an optimization to reduce network traffic
 - $\text{partition}(k_2, \text{number of partitions}) \rightarrow \text{partition for } k_2$
 - ❖ Often a simple hash of the key, e.g., $\text{hash}(k_2) \bmod n$
 - ❖ Divides up key space for parallel reduce operations
- The execution framework handles everything else...

Anything Else?

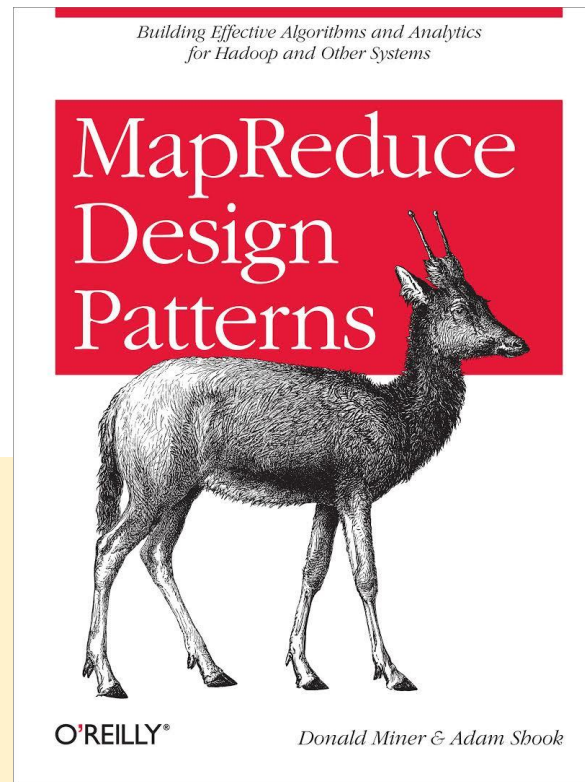


- Handles scheduling
 - Assigns workers to map and reduce tasks
- Handles “data distribution”
 - Moves processes to data
- Handles synchronization
 - Gathers, sorts, and shuffles intermediate data
- Handles errors and faults
 - Detects worker failures and restarts
- Everything happens on top of a distributed file system (HDFS)
- You don’t know:
 - Where mappers and reducers run
 - When a mapper or reducer begins or finishes
 - Which input a particular mapper is processing
 - Which intermediate key a particular reducer is processing



MapReduce

Patterns

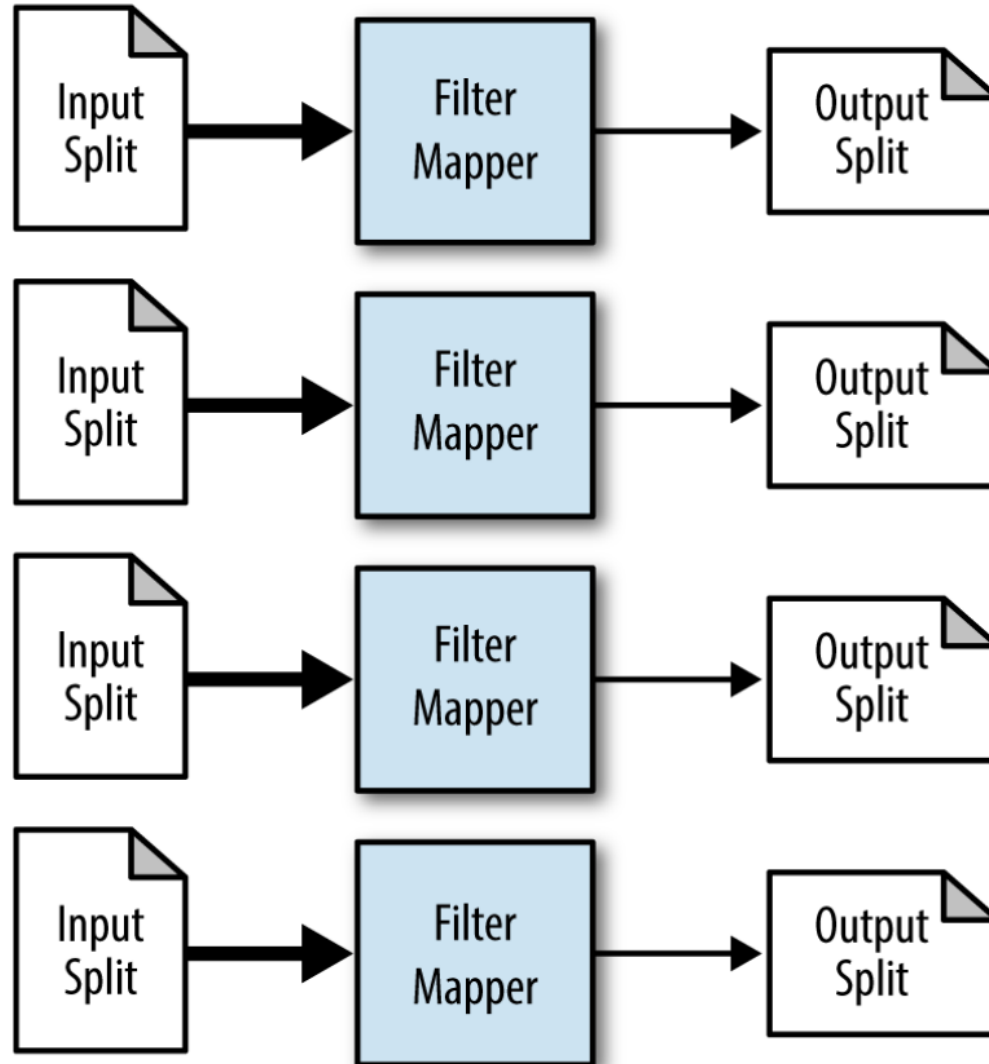


Filtering Pattern



- Goal:
 - Find lines/files/tuples with a particular characteristic
- Examples:
 - Grep Web logs for requests to *.unisa.it/*
 - find in the Web logs the hostnames accessed by 192.168.127.1
 - locate all the files that contain the words 'Apple' and 'Jobs'
- Map: filters and so it does (most of) the work
- Reduce: may simply be the identity

Filtering Pattern



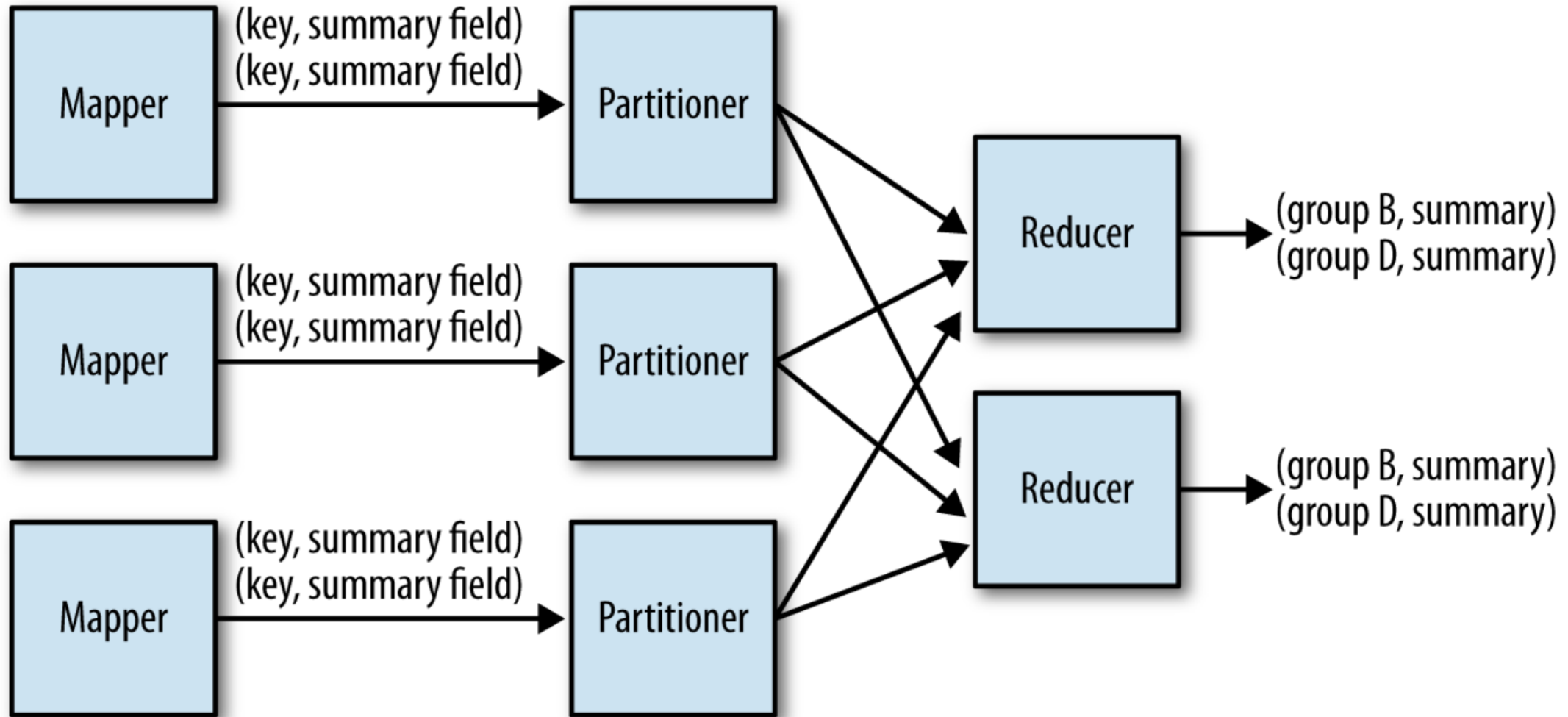
map(key, record):
if we want to keep record then
emit key,value

Summarization Pattern (e.g. WordCounter)



- Goal: Compute the maximum, the sum, the average, ..., over a set of values
- Examples:
 - Count the number of requests to *.unisa.it/*
 - Find the most popular domain
 - Average the number of requests per page per Web site
- Map:
 - filters the input and emits (k, v), where v is a summary value
 - It may be simple or the identity
- Reduce:
 - takes (k, [v]) and applies the summarization to [v]

Summarization Pattern



JOIN Pattern



Stations

Station ID	Station Name
011990-99999	SIHCCAJAVRI
012650-99999	TYNSET-HANSMOEN

Records

Station ID	Timestamp	Temperature
012650-99999	194903241200	111
012650-99999	194903241800	78
011990-99999	195005150700	0
011990-99999	195005151200	22
011990-99999	195005151800	-11

Join

Station ID	Station Name	Timestamp	Temperature
011990-99999	SIHCCAJAVRI	195005150700	0
011990-99999	SIHCCAJAVRI	195005151200	22
011990-99999	SIHCCAJAVRI	195005151800	-11
012650-99999	TYNSET-HANSMOEN	194903241200	111
012650-99999	TYNSET-HANSMOEN	194903241800	78

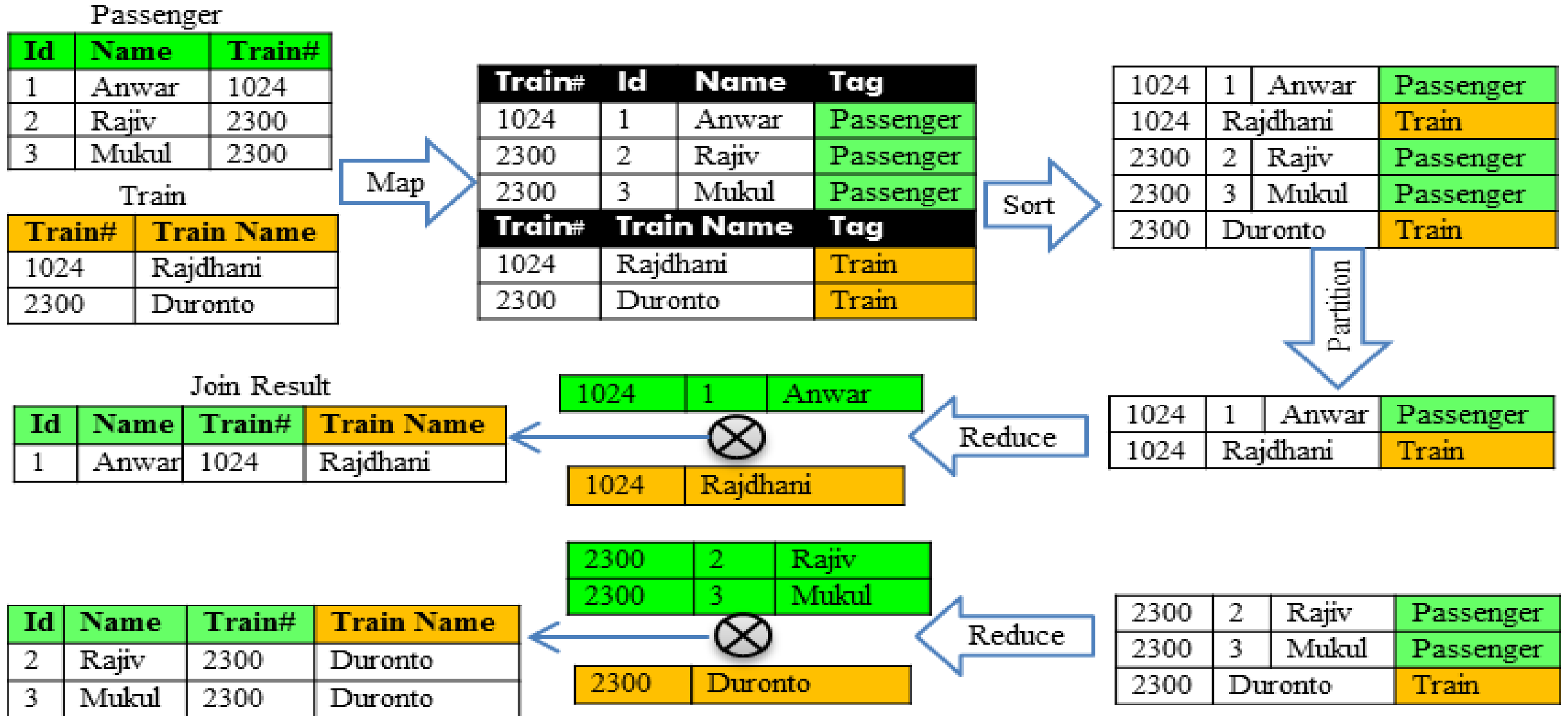
Map Phase :

- Each map task works on a split of either R or L.
- Each map task tags the record with its originating table.
- Outputs the extracted join key and the tagged record as a (key, value) pair.
- The outputs are then partitioned, sorted and merged by the framework.

Reducer Phase :

- All the records for each join key are grouped together and eventually fed to a reducer.
- For each join key, the reduce function first separates and buffers the input records into two sets according to the table tag.
- Performs a cross-product between records in the above sets.

JOIN Pattern

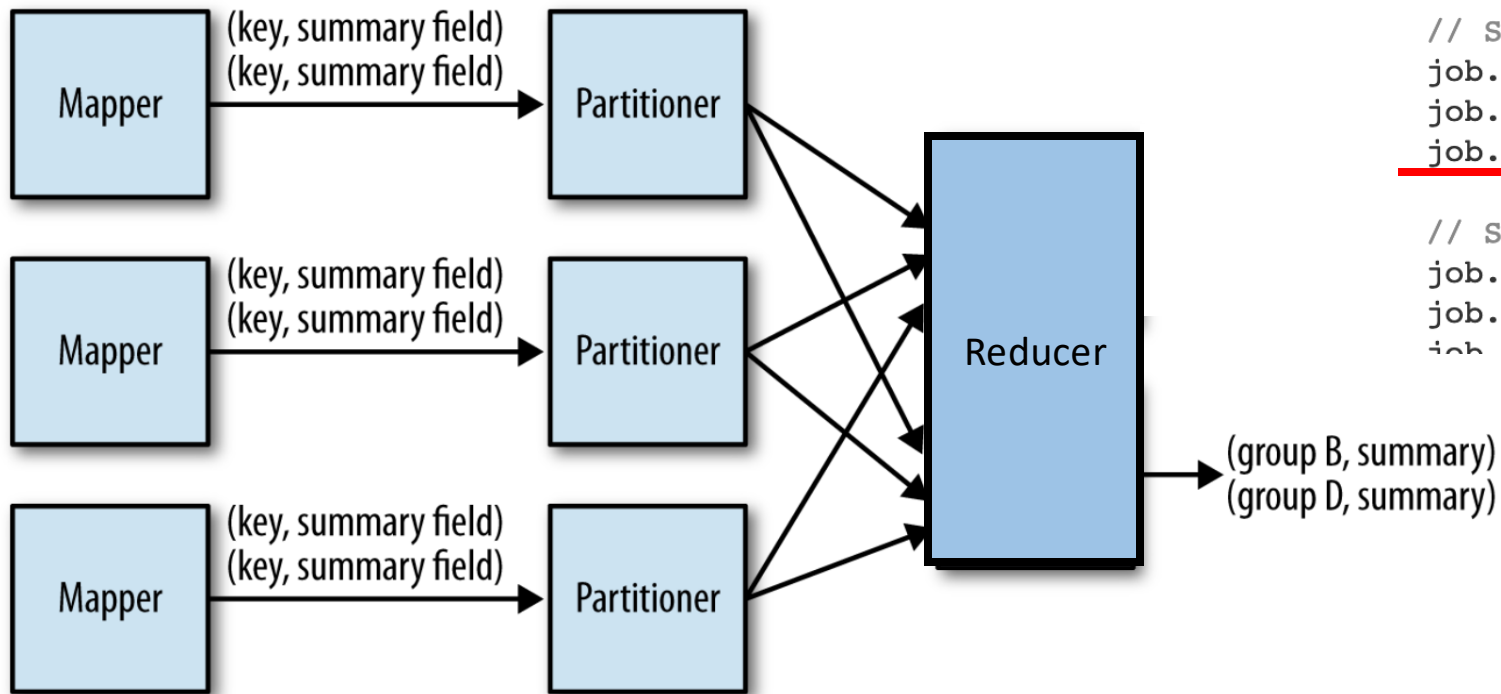


Sorting Pattern



- Goal: Sort input
- Examples:
 - Return all the domains covered by Google's index and the number of pages in each, ordered by the number of pages
- The programming model does not support this per se, but the implementations do
 - The Shuffle stage groups and orders!
- The Map and the Reduce do nothing
- If we have a single reducer, we will get sorted output
- If we have multiple reducers, we can get partly sorted output but it's quite easy to write a last-pass file that merges all of the files
- Take a look here for sorting by value example:
 - <http://santoshsorab.blogspot.com/2014/12/hadoop-java-map-reduce-sort-by-value.html>

Sorting Pattern



```
// Create configuration
Configuration conf = new Configuration(true);

// Create job
Job job = new Job(conf, "Test HIVE command");
job.setJarByClass(ValueSortExp.class);

// Setup MapReduce
job.setMapperClass(ValueSortExp.MapTask.class);
job.setReducerClass(ValueSortExp.ReduceTask.class);
job.setNumReduceTasks(1);

// Specify key / value
job.setMapOutputKeyClass(IntWritable.class);
job.setMapOutputValueClass(IntWritable.class);
job.setOutputKeyClass(IntWritable.class);
```



MapReduce

Hands on MapReduce

Exercises



- Ex0: Take a look at MapReduce WordCount in Python
 - <http://dogdogfish.com/hadoop/hadoop-wordcount-in-python/>
- Ex1: Write a Hadoop MapReduce program which outputs the number of words that start with each letter. This means that for every letter we want to count the total number of words that start with that letter. In your implementation ignore the letter case, i.e., consider all words as lower case. You can ignore all non-alphabetic characters.

Exercise



- How many words start with the same letter?
- Other exercises Python with HadoopStreaming
 - <https://blog.matthewrathbone.com/2016/02/09/python-tutorial.html>

5 min



Advanced Homeworks

Homework 1: NEAR words



- An [n-gram](#) is a tuple of adjacent n words.
- [Google Books Ngram](#) provides counts for every single 1-, 2-, 3-, 4-, and 5-gram observed in the Google Books corpus grouped by year.
- Each row in the n-gram data set is composed of 3 fields: the n-gram, the year, and the number of observations. (You can explore the data interactively [here](#).)
- Exercise: Aggregate the data to count the number of times any pair of words are observed **NEAR** each other, grouped by year.
- “**NEAR**” means that words are observed within 4 words of each other. Or equivalently, two words are near each other if they appear together in **any 2-, 3-, 4-, or 5-gram**. So a row in the resulting data set would be comprised of a **2-gram, a year, and a count**.

Homework1: NEAR words



- A possible solution may be obtained considering 5-grams on google, and then emit for each word pairs (adjacent or not) in the 5-gram the count
- Input
 - (the, cat, in, the, hat) 1999 20
 - (the, cat, is, on, youtube) 1999 13
 - (how, are, you, doing, today) 1986 5000
- Output
 - (the, cat) 1999 33
 - (the, in) 1999 20

Homework 2: People You May Know



- Write a MapReduce program in Hadoop that implements a simple **“People You Might Know”** social network friendship recommendation algorithm. The key idea is that if two people have a lot of mutual friends, then the system should recommend that they connect with each other.
- **Input:** Download the input file from the shared folder 4rdLesson/Homework/Hw1MR/. Input file contains the adjacency list and has multiple lines in the following format: <User><TAB><Friends>
 - <User> is a unique integer ID corresponding to a unique user
 - <Friends> is a comma-separated list of unique IDs corresponding to the friends of the <User>.
- **Algorithm:** Let us use a simple algorithm such that, for each user U, the algorithm recommends N = 10 users who are not already friends with U, but have the largest number of mutual friends in common with U.
- **Output:** The output should contain one line per user in the following format:
<User><TAB><Recommendations>
 - <User> is a unique ID corresponding to a user
 - <Recommendations> is a comma-separated list of unique IDs corresponding to the algorithm’s recommendation of people that <User> might know, ordered by decreasing number of mutual friends.

Homework 2: Notes



- Note that the friendships are mutual (i.e., edges are undirected): if A is friend with B then B is also friend with A. The data provided is consistent with that rule as there is an explicit entry for each side of each edge.
- Even if a user has fewer than 10 second-degree friends, output all of them in decreasing order of the number of mutual friends. If a user has no friends, you can provide an empty list of recommendations. If there are multiple users with the same number of mutual friends, ties are broken by ordering them in a numerically ascending order of their user IDs
- Please provide a description of how you are going to use MapReduce jobs to solve this problem, high-level description of your strategy to tackle this problem.
- Note: It is possible to solve this question with a single MapReduce job. But if your solution requires multiple MapReduce jobs, then that's fine too.